Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM

Deepak Narayanan‡ ★, Mohammad Shoeybi†, Jared Casper†, Patrick LeGresley†, Mostofa Patwary†, Vijay Korthikanti†, Dmitri Vainbrand†, Prethvi Kashinkunti†, Julie Bernauer†, Bryan Catanzaro†, Amar Phanishayee∗, Matei Zaharia‡

†*NVIDIA* ‡*Stanford University* ∗*Microsoft Research*

Overview

- Recent work [11, 40] has shown large language models to be effective zero- or few-shot learners (e.g., able to categorize data into classes not seen during training), with high accuracy on many NLP tasks and datasets.
- These large language models have a number of downstream applications such as client feedback summarization, automatic dialogue generation, semantic search, and code autocompletion [1, 4, 5].
- The number of parameters in state-of-the-art NLP models have grown at an exponential rate (Figure 1). Training such models, however, is challenging as it is no longer possible to fit the parameters of these models in the main memory of a single GPU, and in some extreme cases, multiple GPUs on a server.
- Training GPT-3 with 175 billion parameters [11] would require approximately 288 years with a single V100 NVIDIA GPU.

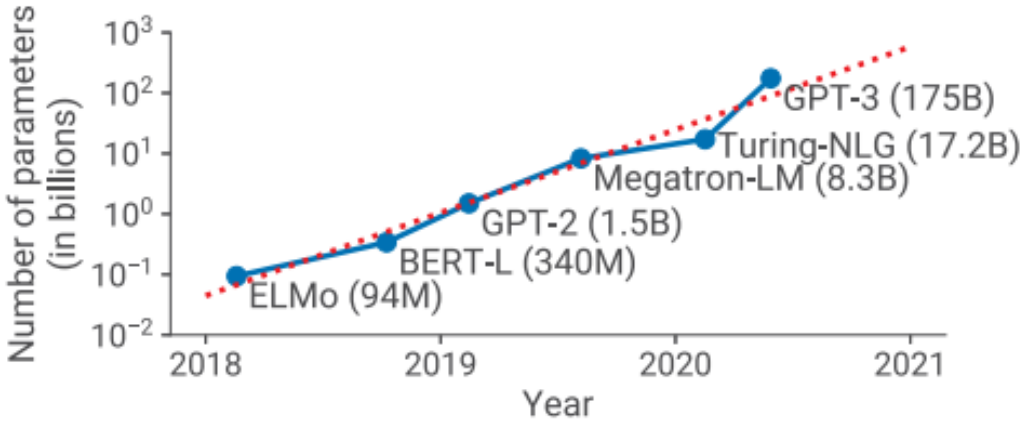


Figure 1: Trend of sizes of state-of-the-art Natural Language Processing (NLP) models with time. The number of floating-point operations to train these models is increasing at an exponential rate.

Overview

- In this paper, we address the following question:

How should parallelism techniques be combined to maximize the training throughput of large models given a batch size while retaining strict optimizer semantics?

- Our method leverages the combination of
  - Pipeline parallelism across multi-GPU servers,
  - Tensor parallelism within a multi-GPU server, and
  - Data parallelism
- To practically train models with a trillion parameters,
- Graceful scaling in an optimized cluster environment with high-bandwidth links between GPUs on the same server and across servers.

Overview

Contribution summary

- In this paper, we show how tensor, pipeline, and data parallelism can be composed to scale to thousands of GPUs. The technique is called PTD-P.
- We propose a novel interleaved pipelining schedule that can improve throughput by 10+% with memory footprint comparable to existing approaches.
- Our approach allows us to perform training iterations on a model with
  - 1 trillion parameters at 502 petaFLOP/s on 3072 GPUs on a GPT model (per-GPU throughput of 52% of theoretical peak).
  - End-to-end training throughput of 163 teraFLOP/s per GPU (including communication, data processing, and optimization).
- We also compared to ZeRO [36], and found that our approach outperforms ZeRO-3 by 70% for models with 175 and 530 billion parameters due to less cross-node communication. These models are too large to fit on a multi-GPU server.

Overview

Contribution summary

- The following are ideas that are accelerator-agnostic:
  - Smartly partitioning the model training graph to minimize the amount of communication while still keeping devices active.
  - Minimizing the number of memory bound kernels with operator fusion and careful data layout.

Limitations of exiting techniques

- Data-parallel, scale-out solutions usually works well, but suffers from two limitations:
  - a) beyond a point, the per-GPU batch size becomes too small, reducing GPU utilization and increasing communication cost, and
  - b) the maximum number of devices that can be used is the batch size, limiting the number of accelerators that can be used for training.
- Recent work [39, 40] has shown how tensor (intra-layer) model parallelism, where matrix multiplications within each transformer layer are split over multiple GPUs, can be used to overcome these limitations.
- Tensor parallelism breaks down for larger models. Larger models need to be split across multiple multi-GPU servers, which leads to two problems:
  - (a) the all-reduce communication required for tensor parallelism needs to go through inter-server links, which are slower than the high-bandwidth NVLink [9] available within a multi-GPU server, and
  - (b) a high degree of model parallelism can create small matrix multiplications (GEMMs), potentially decreasing GPU utilization.
- Pipeline model parallelism [14, 20, 23, 29, 30, 45] is another technique to support the training of large models, where layers of a model are striped over multiple GPUs.
  - To preserve strict optimizer semantics, optimizer steps need to be synchronized across devices, leading to a *pipeline flush* at the end of every batch (pipeline bubbles). As much as 50% of time can be spent flushing the pipeline depending on the number of microbatches injected into the pipeline.
  - To achieve high efficiency, a larger batch size is often necessary. In this work, we also introduce a new pipeline schedule that improves efficiency at small batch sizes.

Observations

- Even with high-bandwidth network links between servers
  - Tensor model parallelism is effective within a multi-GPU server,
  - Pipeline model parallelism must be used for larger models.
  - The schedule used for pipeline parallelism has an impact on the amount of communication, the pipeline bubble size, and memory used to store activations.
- Values of hyperparameters such as microbatch size have an impact on
  - The memory footprint,
  - The arithmetic efficiency of kernels executed on the worker, and
  - the pipeline bubble size.
  - The optimal value of the microbatch size is problem-dependent.
- At scale, distributed training is communication-intensive.
  - When training a trillion-parameter model on 3072 GPUs, our implementation used an effective bisection bandwidth of 892 GB/s for pipeline-parallel communication, and 13 TB/s for data-parallel communication.
  - Using slower inter-node interconnects or more communication-intensive partitioning would hinder scaling performance.
- Related work such as FlexFlow [22], PipeDream [29], Tarnawski et al. [41], and DAPPLE [14] automatically explore the search space of parallelism strategies, but instead
  - We suggest heuristics (in §3) that we found work well in practice.

Parallelism techniques

Data parallelism

- Each worker has a copy of the full model, the input dataset is sharded, and workers aggregate their gradients periodically to ensure that all workers see a consistent version of the weights.
- For large models which do not fit on a single worker, data parallelism can be used on smaller model shards.

Parallelism techniques

Pipeline parallelism

- When used on models with the same transformer block repeated, each device can be assigned an equal number of transformer layers.
- We do not consider more asymmetric model architectures, where assignment of layers to pipeline stages is harder; we defer to related work [22, 29, 41] to solve this problem.
- A batch is split into smaller microbatches; execution is then pipelined across microbatches.
- Pipelining schemes need to ensure that inputs see consistent weight versions across forward and backward passes for well-defined synchronous weight update semantics.
- Naïve pipelining can lead to an input seeing weight updates in the backward pass not seen in the forward pass.

Parallelism techniques

- To retain strict optimizer semantics *exactly*, we introduce periodic pipeline flushes so that optimizer steps are synchronized across devices.
- At the start and end of every batch, devices are idle. We call this idle time the *pipeline bubble*, and want to make it as small as possible.
- Asynchronous and bounded-staleness approaches such as PipeMare, PipeDream, and PipeDream-2BW [23, 29, 30, 45] do away with flushes completely, but relax weight update semantics. We defer consideration of such schemes to future work.

Parallelism techniques

Default pipeline schedule

- Gpipe [20] proposes a schedule where the forward passes for all microbatches in a batch are first executed, followed by backward passes for all microbatches.
- We denote the size of GPipe's pipeline bubble as t_{pb}
- The number of microbatches in a batch as m
- The number of pipeline stages (number of devices used for pipeline parallelism) as p
- The ideal time per iteration as t_{id} (assuming perfect or ideal scaling)
- The time to execute a single microbatch's forward and backward pass as t_f and t_b, respectively
- The pipeline bubble consists of p - 1 forward passes at the start of a batch and p - 1 backward passes at the end

- The total amount of time spent in the pipeline bubble is then $t_{pb} = (p-1) \cdot (t_f + t_b)$
- The ideal processing time for the batch is $t_{id} = m \cdot (t_f + t_b)$
- The fraction of ideal computation time spent in the pipeline bubble is (plugin the terms from before)

$$\text{Bubble time fraction (pipeline bubble size)} = \frac{t_{pb}}{t_{id}} = \frac{p-1}{m}.$$

Parallelism techniques

Default pipeline schedule

- For the bubble time fraction to be small, we thus need $m \gg p$.
- However, for such large m, this approach has a high memory footprint as it requires stashed intermediate activations (or just input activations for each pipeline stage when using activation recomputation) to be kept in memory for all m microbatches through the lifetime of a training iteration.
- Instead, we use the PipeDream-Flush schedule (see the next slide).

Parallelism techniques

Schedule with interleaved pipeline stages

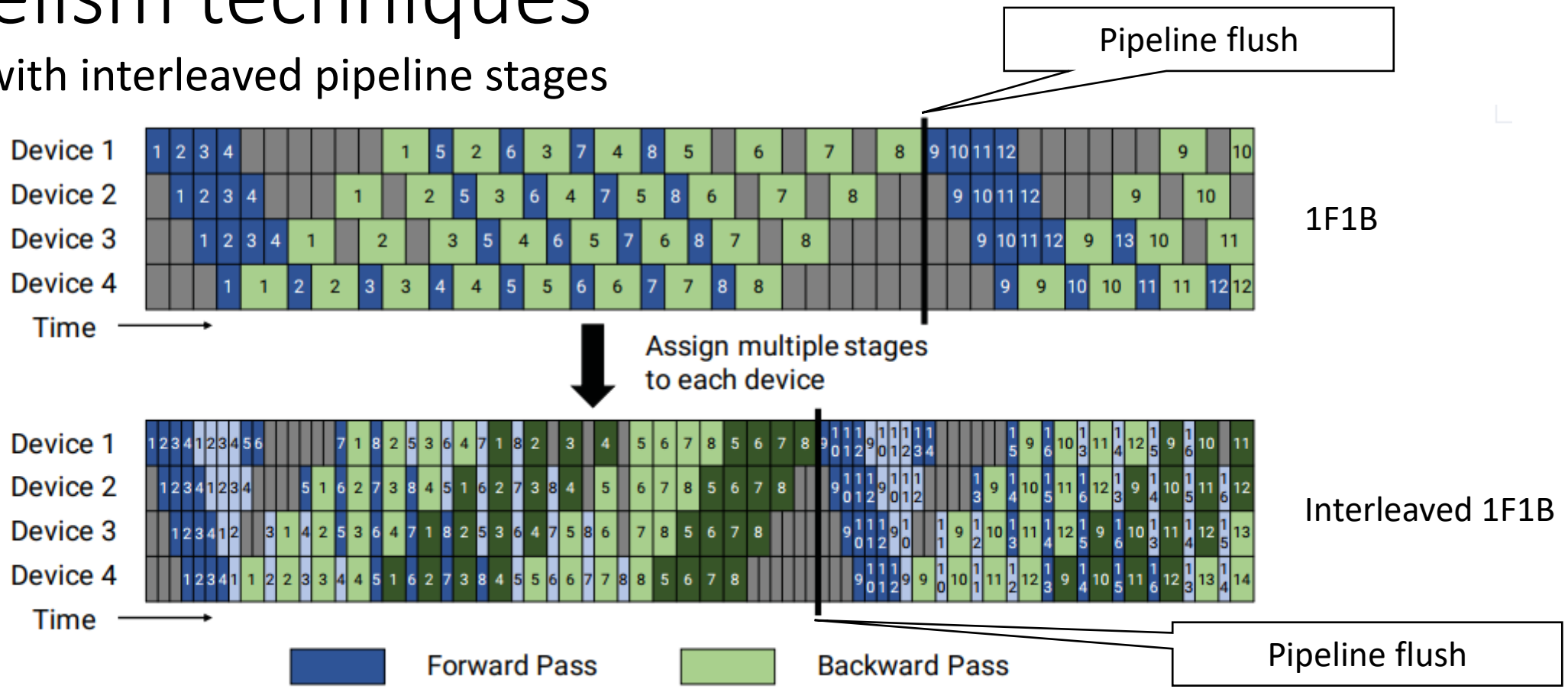


Figure 4: Default and interleaved 1F1B pipeline schedules. The top figure shows the default non-interleaved 1F1B schedule. The bottom figure shows the interleaved 1F1B schedule, where each device is assigned multiple chunks (in this case, 2). Dark colors show the first chunk and light colors show the second chunk. The size of the pipeline bubble is smaller (the pipeline flush happens sooner in the interleaved timeline).

Parallelism techniques

Schedule with interleaved pipeline stages

- To reduce the size of the pipeline bubble, each device can perform computation for multiple subsets of layers (called a model chunk), instead of a single contiguous set of layers.
- The pipeline flush for the same batch size happens sooner in the new schedule.
- As before, we can use an "all-forward, all-backward" version of this schedule, but this has a high memory footprint (proportional to m, number of microbatches in a batch).
- If each device has v stages (or model chunks), then the forward and backward time for a microbatch for each stage or chunk will now be t_f / v and t_b / v.
- This means that the new schedule reduces the bubble time by v.
- This reduced pipeline bubble size, however, does not come for free: this schedule requires extra communication. Quantitatively, the amount of communication also increases by v.

The pipeline bubble time thus reduces to $t_{pb}^{\text{int.}} = \frac{(p-1)\cdot(t_f+t_b)}{v}$, and the bubble time fraction is then:

$$\text{Bubble time fraction (pipeline bubble size)} = \frac{t_{pb}^{\text{int.}}}{t_{id}} = \frac{1}{v} \cdot \frac{p-1}{m}.$$

Recall

$t_{pb} = (p\text{-}1) \cdot (t_f + t_b)$

$t_{id} = m \cdot (t_f + t_b))$

(see Default schedule slides)

Parallelism techniques

Tensor model parallelism

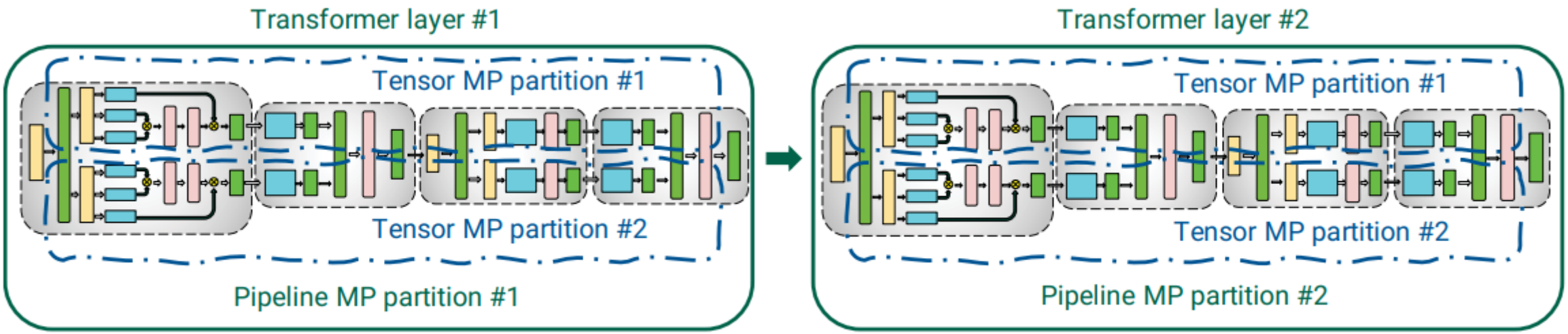
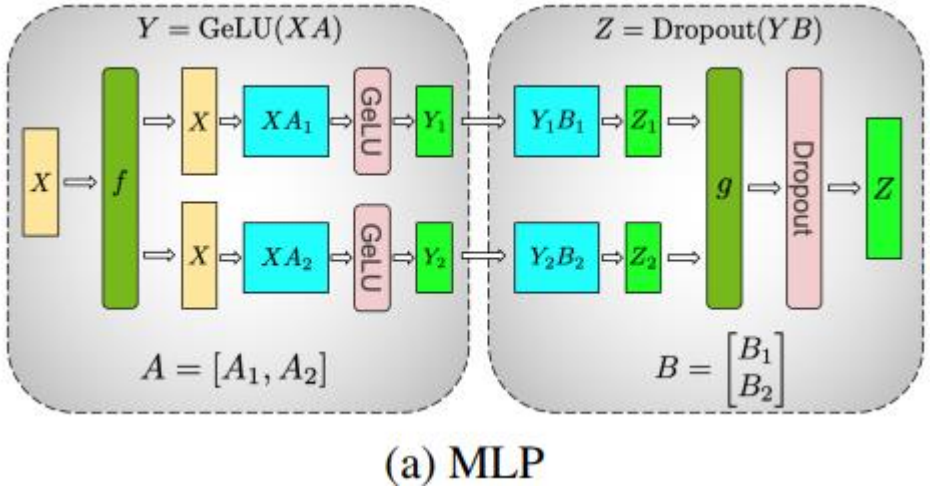


Figure 2: Combination of tensor and pipeline model parallelism (MP) used in this work for transformer-based models.

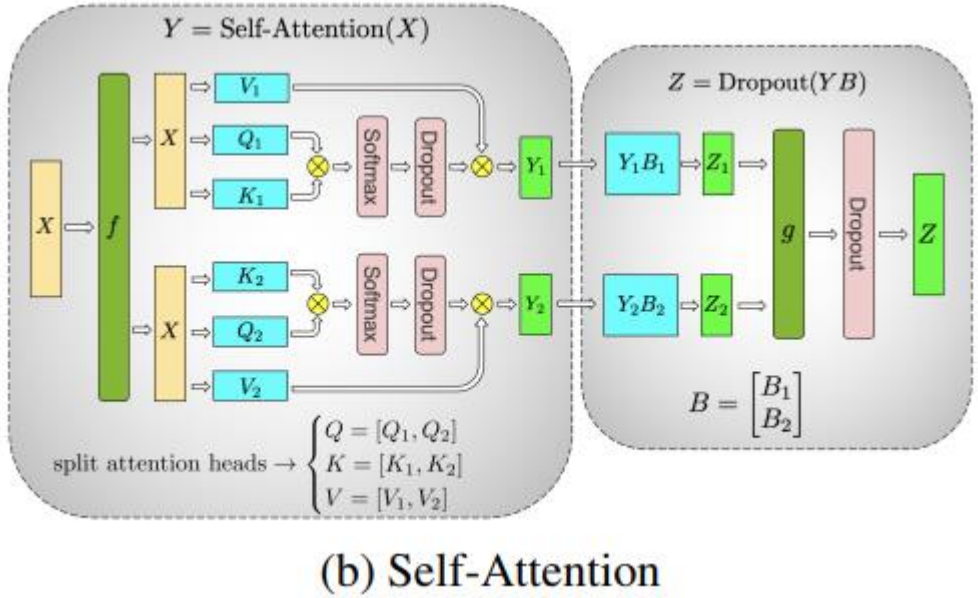
Parallelism techniques

Tensor model parallelism

- Individual layers of the model are partitioned over multiple devices.
- In this paper, we use the particular partitioning strategy used by Megatron [40] for transformer layers. (Skip the description in the rest of the subsection. Understanding of general tensor model parallelism is enough.)



(a) MLP



(b) Self-Attention

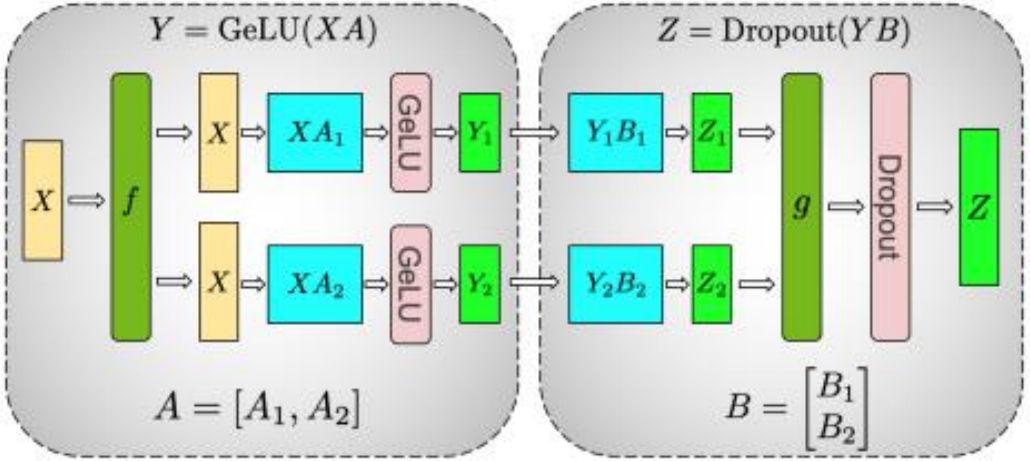
Figure 5: Blocks of transformer model partitioned with tensor model parallelism (figures borrowed from Megatron [40]). f and g are conjugate. f is the identity operator in the forward pass and all-reduce in the backward pass, while g is the reverse.

f and g are all-reduce operators applied to partitioned data.
$\otimes$ is the tensor product operator.

Parallelism techniques

Tensor model parallelism

- A transformer layer consists of a self-attention block followed by a two-layer multi-layer perceptron (MLP).
- The MLP block consists of two GEMMs and a GeLU non-linearity: $Y = \mathrm{GeLU}(XA). \quad Z = \mathrm{Dropout}(YB).$
- We can split A along its columns $A = [A1, A2]$.
- This partitioning allows the GeLU non-linearity to be independently applied to the output of each partitioned GEMM: $[Y_1, Y_2] = [\mathrm{GeLU}(XA_1), \mathrm{GeLU}(XA_2)].$
- This is advantageous as it removes the need for synchronization (needed if A is split along its rows since GeLU is non-linear).



(a) MLP

Figure 5: Blocks of transformer model partitioned with tensor model parallelism (figures borrowed from Megatron [40]). f and g are conjugate. f is the identity operator in the forward pass and all-reduce in the backward pass, while g is the reverse.

f and g are all-reduce operators applied to partitioned data.

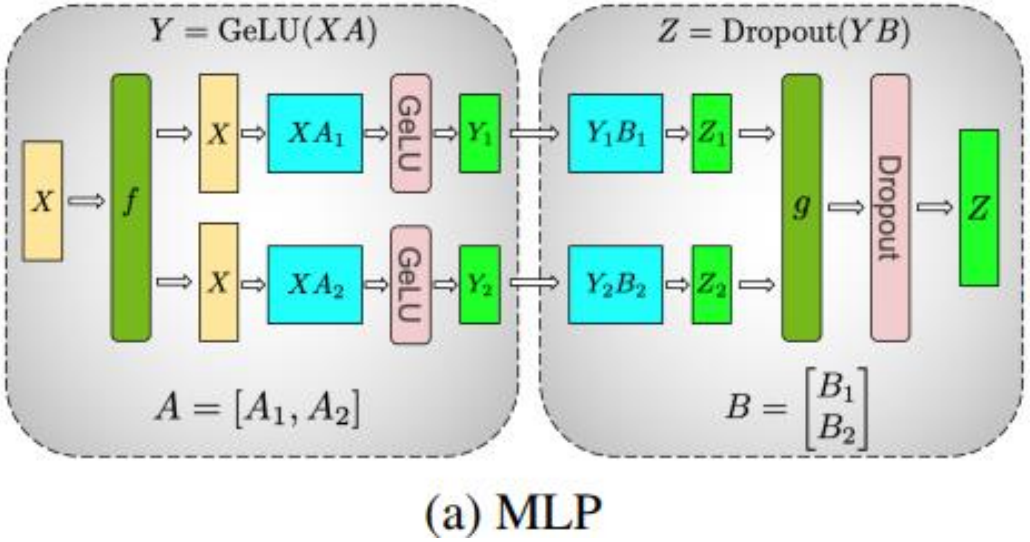
Parallelism techniques

Tensor model parallelism

- The rows of the second weight matrix B can then be split along its rows to remove the need for any communication between the GEMMs (shown in Figure 5a), as shown below:

$$B = \begin{bmatrix} B_1 \\ B_2 \end{bmatrix}, \quad Y = [Y_1, Y_2].$$

- The output of the second GEMM is then reduced across the GPUs before the dropout layer.



(a) MLP

Figure 5: Blocks of transformer model partitioned with tensor model parallelism (figures borrowed from Megatron [40]). f and g are conjugate. f is the identity operator in the forward pass and all-reduce in the backward pass, while g is the reverse.

f and g are all-reduce operators applied to partitioned data.

Parallelism techniques

Tensor model parallelism

- We exploit the inherent parallelism in the multi-head attention operation to partition the self-attention block (shown in Figure 5b).
- The key (K), query (Q), and value (V) matrices can be partitioned in a column-parallel fashion.
- The output linear layer can then directly operate on the partitioned output of the attention operation (weight matrix partitioned across rows).
- This approach splits GEMMs in the MLP and self-attention blocks across GPUs while requiring only two all-reduce operations in the forward pass (g operator) and two all-reduces in the backward pass (f operator).

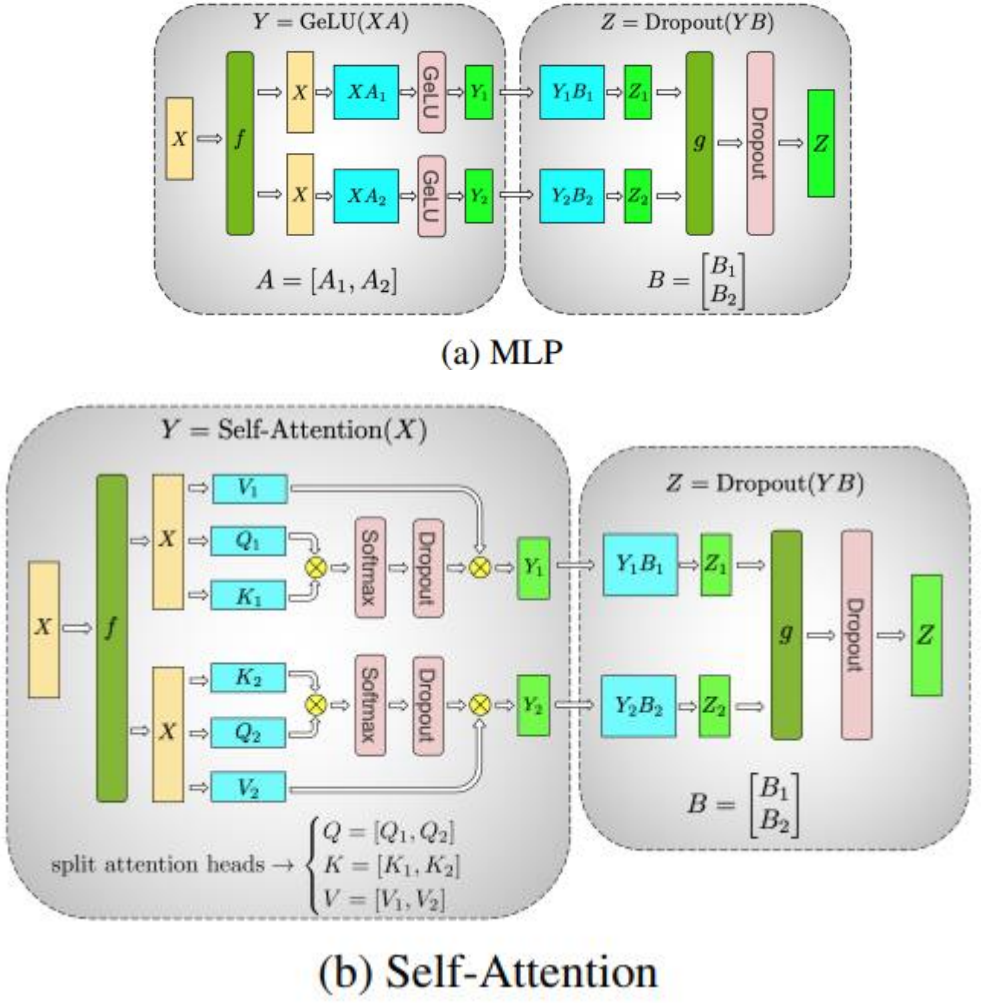
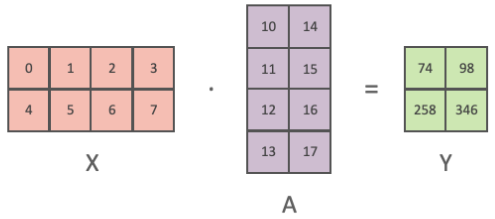


Figure 5: Blocks of transformer model partitioned with tensor model parallelism (figures borrowed from Megatron [40]). f and g are conjugate. f is the identity operator in the forward pass and all-reduce in the backward pass, while g is the reverse.

f and g are all-reduce operators applied to partitioned data.
$\otimes$ is the tensor product operator.

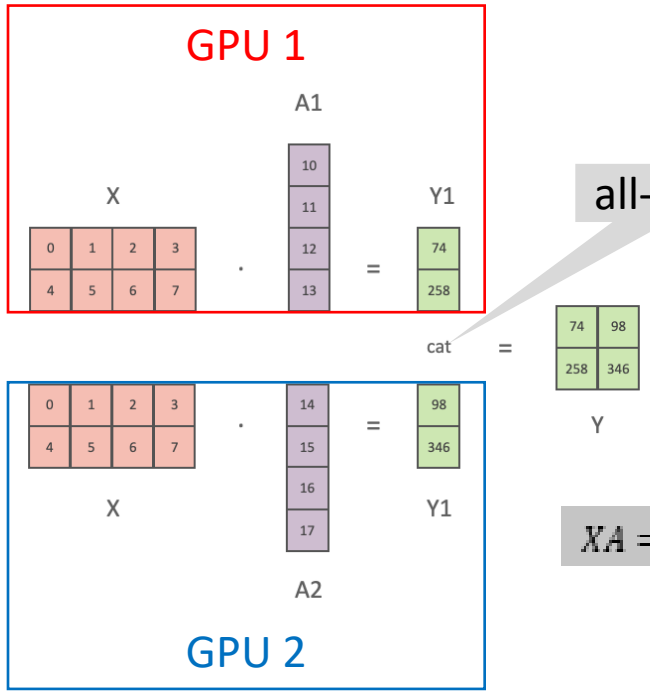
Tensor parallelism

- A common operation in deep neural networks is *XA* = *Y*



is equal to

column parallelism

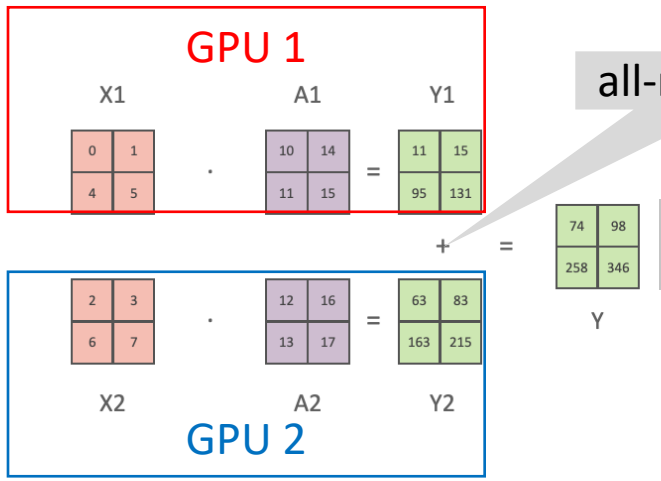


all-gather

$$XA = X[A_1\ A_2] = [XA_1\ XA_2] = [Y_1\ Y_2] = Y$$

Note: *Y* is available on both GPUs

row parallelism



all-reduce(sum)

$$XA = [X_1\ X_2]\begin{bmatrix} A_1 \\ A_2 \end{bmatrix} = X_1A_1 + X_2A_2 = Y_1 + Y_2 = Y$$

Figure source: https://huggingface.co/transformers/v4.12.5/parallelism.html

Performance modeling

- We study performance implications of combining pipeline and tensor model parallelism with data parallelism.
- Given a fixed budget of GPUs and batch size, one can use different degrees of the parallelism types in PTD-P to train models.
- Each dimension exposes tradeoffs between
  - Memory footprint,
  - Device utilization, and
  - Amount of communication.
- We present analytical models
  - For the pipeline bubble size.
  - We qualitatively describe how communication time behaves and present cost models for amount of communication.
  - We do not present direct cost models for communication time, which is harder to model for a hierarchical network topology.

Performance modeling

3.1 Notation

We use the following notation in this section:

- (p, t, d): Parallelization dimensions. p for the pipeline-model-parallel size, t for the tensor-model-parallel size, and d for the data-parallel size.
- n: Number of GPUs. We require $p \cdot t \cdot d = n$.
- B: Global batch size (provided as input).
- b: Microbatch size.
- $m = \frac{1}{b} \cdot \frac{B}{d}$: Number of microbatches in a batch *per pipeline*.

Performance modeling

Tensor and pipeline model parallelism

- Tensor and pipeline model parallelism can both be used to partition a model's parameters over multiple GPUs.
- using pipeline parallelism with periodic flushes results in a pipeline bubble of size $(p - 1)/m$ (described earlier).
- Let us assume that $d = 1$ (data-parallel size); consequently, $t \cdot p = n$. The pipeline bubble size in terms of t (tensor-model-parallel size) is:

$$\frac{p-1}{m} = \frac{n/t - 1}{m}.$$

- As t increases, the pipeline bubble thus decreases for fixed B, b, and d . Here, number of microbatches per pipeline, $m = B/(b \cdot d)$ is fixed as well.

Performance modeling

Tensor and pipeline model parallelism

- The amount of communication performed between different GPUs is also affected by the values of p and t.
- Pipeline model parallelism features cheaper point-to-point communication. The total amount of communication that needs to be performed between every pair of consecutive devices (for either the forward or backward pass) for each microbatch is bsh.
  - Here, b is the microbatch size, s is the sequence length and h is the size of the hidden layer (number of parameters).
- Tensor model parallelism, on the other hand, uses all-reduce communication (two all-reduce operations each in the forward and backward pass, see §2.3).
- Tensors of total size bsh need to be all-reduced among t model replicas twice each in the forward and backward pass for each layer, leading to a total communication of per layer per device for each microbatch:
- $$8bsh\left(\frac{t-1}{t}\right)$$
- Each device typically has multiple layers; the total amount of tensor-parallel-communication per device for each microbatch, given l^{stage} is the number of layers in a pipeline stage, is then:
- $$l^{\text{stage}} \cdot \left(8bsh\left(\frac{t-1}{t}\right)\right)$$
- We see that tensor model parallelism increases the amount of communication between devices. Thus, when t is larger than the number of GPUs in a single node, the overhead of performing tensor model parallelism across slower inter-node links can be impractical.
- **Takeaway #1:** When considering different forms of model parallelism, tensor model parallelism should generally be used up to degree g when using g-GPU servers, and then pipeline model parallelism can be used to scale up to larger models across servers.

Performance modeling

Data and pipeline model parallelism

- We consider the interaction between data parallelism and pipeline model parallelism.
- Let $t = 1$ (tensor-model-parallel size).
- The number of microbatches per pipeline is $m = B/(d \cdot b) = b'/d$, where $b' := B/b$.
- With total number of GPUs n, the number of pipeline stages is $p = n/(t \cdot d) = n/d$. The pipeline bubble size is:

$$\frac{p-1}{m} = \frac{n/d-1}{b'/d} = \frac{n-d}{b'}.$$

- As d becomes larger, the term n - d becomes smaller, and thus the pipeline bubble becomes smaller.
- It might not be possible to increase d all the way to n for all models, since a model's full training memory footprint might be larger than the memory capacity of a single GPU.

Performance modeling

Data and pipeline model parallelism

- Overall throughput will thus increase if the all-reduce communication needed for data parallelism does not drastically increase with higher d, which should hold since the communication time for a ring-based implementation scales with

$$\frac{d-1}{d} = 1 - \frac{1}{d}.$$

- Recall the ring algorithm requires $O(d-1)$ communication steps.
- We can also analyze the impact of increasing the batch size B:
  - For a given parallel configuration, as the batch size B increases, $b' = B/b$ increases, $(n - d)/b'$ decreases, consequently increasing throughput.
  - All-reduce communication required by data parallelism also becomes more infrequent, further increasing throughput.

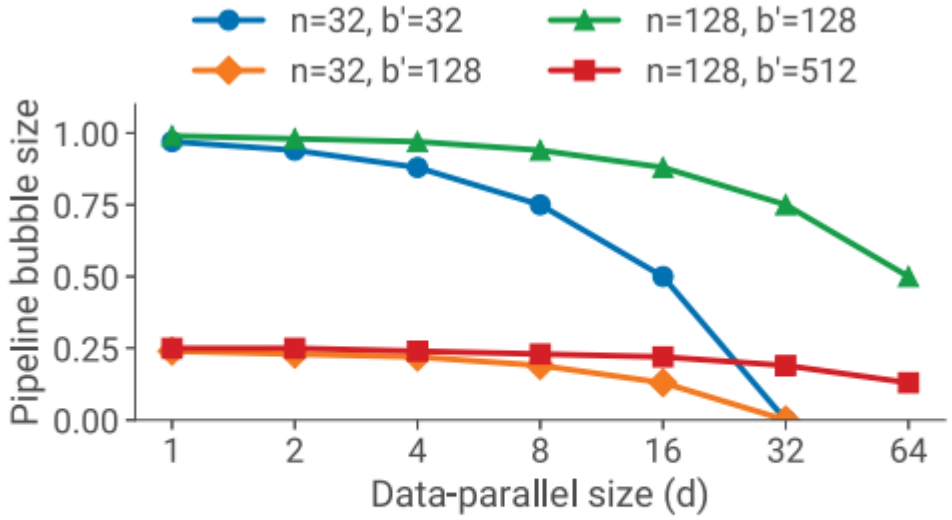


Figure 6: Fraction of time spent idling due to pipeline flush (pipeline bubble size) versus data-parallel size (d), for different numbers of GPUs (n) and ratio of batch size to microbatch size ($b' = B/b$).

Performance modeling

Data and tensor model parallelism

- With tensor model parallelism, all-reduce communication needs to be performed for *every* microbatch (since individual layers are partitioned). This can be expensive across multi-GPU servers (requires inter-server communication over the network).
- Moreover, with tensor model parallelism, each model-parallel rank performs a subset of the computation in each model layer, and thus for insufficiently large layers, modern GPUs might not perform these sub-matrix computations with peak efficiency (i.e., GPU resources could be under utilized).
- On the other hand, data parallelism only needs to perform expensive all-reduce communication *once per batch*.
- **Takeaway #2:** When using data and model parallelism, a total model-parallel size of $M = t \cdot p$ should be used so that the model's parameters and intermediate metadata fit in GPU memory; data parallelism can be used to scale up training to more GPUs.

Performance modeling

What is a good microbatch size?

- We now want to determine the optimal microbatch size b given a parallel configuration (p, t, d) and batch size B.
- The amount of data-parallel communication will be the same regardless of the microbatch size.
- Given functions $t_f(b)$ and $t_b(b)$ that map the microbatch size to the forward and backward computation times for a single microbatch, respectively, the total time spent computing a batch, ignoring communication cost, is (as before, define b' as B/d):

$$(b'/b + p - 1) \cdot \left(t_f(b) + t_b(b)\right).$$

Earlier we computed the amount of time spent in the pipeline bubble is $t_{pb} = (p\text{-}1) \cdot (t_f + t_b)$

- The microbatch size thus affects both the arithmetic intensity of operations as well as the pipeline bubble size (by affecting m).
- **Takeaway #3:** The optimal microbatch size b depends on the throughput and memory footprint characteristics of the model, as well as the pipeline depth p, data-parallel size d, and batch size B.

.

Performance modeling

Activation recomputation

- Activation recomputation [12, 18, 20, 21] is an optional technique that trades off an increase in the number of compute operations performed for additional memory footprint.
- By running the forward pass a second time just before the backward pass and stashing only the input activations for a given pipeline stage, as opposed to the entire set of intermediate activations, which is much larger.
- The number of activation checkpoints does not impact throughput (GPU FLOP/s achieved), but impacts memory footprint.

Evaluation

Questions to be answered

- How well does PTD-P perform? Does it result in realistic end-to-end training times?
- How well does pipeline parallelism scale for a given model and batch size?
- How much impact does the interleaved schedule have on performance?
- How do different parallelization dimensions interact with each other?
- What is the impact of hyperparameters such as microbatch size?
- What is the impact of the scatter-gather communication optimization?
- What types of limits do we put on hardware when running training iterations at scale?

Evaluation

- The testbed is Selene GPU cluster [8].
- Each cluster node has 8 NVIDIA 80-GB A100 GPUs [6], connected to each other by NVLink and NVSwitch [9].
- Each cluster node has 8 Mellanox 200Gbps HDR Infiniband HCAs (network interface card) for application communication, with an additional 2 HCAs per node for dedicated storage.
- The peak device throughput of an A100 GPU with 16-bit precision is 312 teraFLOP/s.
- The nodes are connected in a three-level (leaf, spine, core) fat-tree topology with 850 switches. This topology allows efficient all-reduce communication (dominant communication pattern in deep learning training).

Evaluation

Methodology

- For most of our results, we report throughput per GPU. Aggregate throughput can be computed by multiplying with the number of GPUs used.
- We use GPT LLM models of appropriate sizes. In particular, for any given microbenchmark (including input dataset), the model needs to fit on the number of model-parallel GPUs used in the experiment.
- We use standard model architectures such as GPT-3 [11] when appropriate.
- All models use a vocabulary size (denoted by V) of 51,200 (multiple of 1024) and a sequence length (s) of 2048 (#tokens, i.e., words in a single input/output).
- Experiments use tensor, pipeline, and data parallelism (degrees picked using heuristics described in §3, performance modeling). In particular, we use the interleaved pipeline schedule with the scatter/gather optimization enabled.

Evaluation

End-to-end performance

- We vary hidden size (h), number of attention heads, and number of layers (l). The number of parameters in a model, P, can be computed as:

$$P = 12lh^2\left(1 + \frac{13}{12h} + \frac{V+s}{12lh}\right).$$

- As the model size increases, we also increase the batch size (B) and the number of GPUs (n).
- The majority of floating-point operations in the model are performed in the matrix multiplications (GEMMs) in the transformer and logit layers (final layer output usually fed to an activation function).
- Considering just these GEMMs, the number of FLOPs per iteration is (more details in the Appendix):

$$F = 96Bslh^2\left(1 + \frac{s}{6h} + \frac{V}{16lh}\right).$$

- This is a lower bound for the true FLOP count but should be close to the actual value. We count a FLOP as a floating-point operation regardless of precision.

Evaluation

End-to-end performance

- Throughput is measured for end-to-end training, i.e., includes all operations including data loading, optimizer steps, communication, and logging.
- Training time estimates (training on T tokens): Training requires $I = T/(B \cdot s)$ iterations. Using the value of F (in the previous slide) and empirical end-to-end throughputs from Table 1 (denoted by X), we can estimate total training time.
- For the configurations in Table 1, we have $6h \gg s$, $16lh \gg (V + s)$, and $12lh \gg V$. Combining these observations with equations in the previous slide, we arrive at

$$\text{End-to-end training time} \approx \frac{8TP}{nX}.$$

- Example: Let us consider the GPT-3 model with P=175 billion parameters as an example. This model was trained on T = 300 billion tokens. On n = 1024 A100 GPUs using batch size 1536, we achieve X = 140 teraFLOP/s per GPU. As a result, the time required to train this model is 34 days. For the 1 trillion parameter model, we assume that 450 billion tokens are needed for end-to-end training. With 3072 A100 GPUs, we can achieve a per-GPU throughput of 163 teraFLOP/s, and end-to-end training time of 84 days. We believe these training times (using a reasonable number of GPUs) are practical.
- Estimated time of around 3 months for a trillion-parameter model.

Evaluation

End-to-end performance

Number of parameters (billion)	Attention heads	Hidden size	Number of layers	Tensor model-parallel size	Pipeline model-parallel size	Number of GPUs	Batch size	Achieved teraFlOP/s per GPU	Percentage of theoretical peak FLOP/s	Achieved aggregate petaFLOP/s
1.7	24	2304	24	1	1	32	512	137	44%	4.4
3.6	32	3072	30	2	1	64	512	138	44%	8.8
7.5	32	4096	36	4	1	128	512	142	46%	18.2
18.4	48	6144	40	8	1	256	1024	135	43%	34.6
39.1	64	8192	48	8	2	512	1536	138	44%	70.8
76.1	80	10240	60	8	4	1024	1792	140	45%	143.8
145.6	96	12288	80	8	8	1536	2304	148	47%	227.1
310.1	128	16384	96	8	16	1920	2160	155	50%	297.4
529.6	128	20480	105	8	35	2520	2520	163	52%	410.2
1008.0	160	25600	128	8	64	3072	3072	163	52%	502.0

Table 1: Weak-scaling throughput for GPT models ranging from 1 billion to 1 trillion parameters.

Evaluation

End-to-end performance

APPENDIX: FLOATING-POINT OPERATIONS

In this section, we describe how we calculate the number of floating-point operations (FLOPs) in a model. We consider a language model with l transformer layers, hidden size h, sequence length s, vocabulary size V, and training batch size B.

A $A_{m\times k} \times X_{k\times n}$ matrix multiplication requires $2m \times k \times n$ FLOPs (factor of 2 needed to account for multiplies and adds).

A transformer layer consists of an attention block followed by a 2-layer feed-forward network. For the attention block, the main FLOP contributors are the key, query, and value transformation ($6Bsh^2$ operations), attention matrix computation ($2Bs^2h$ operations), attention over values ($2Bs^2h$ operations), and post-attention linear projection ($2Bsh^2$ operations). The feed-forward network increases the hidden size to $4h$ and then reduces it back to h; this requires $16Bsh^2$ FLOPs. Summing these together, each transformer layer results in $24Bsh^2 + 4Bs^2h$ FLOPs for the forward pass. The backward pass requires double the number of FLOPs since we need to calculate the gradients with respect to both input and weight tensors. In addition, we are using activation recomputation, which requires an additional forward pass before the backward pass. As a result, the total number of FLOPs per transformer layer is $4 \times \left(24Bsh^2 + 4Bs^2h\right) = 96Bsh^2\left(1 + \frac{s}{6h}\right)$.

The other main contributor to the FLOP count is the logit layer in the language model head, which transforms features of dimension h to the vocabulary dimension V. The required FLOPs for this operation is $2BshV$ in the forward pass and $4BshV$ in the backward pass, resulting in $6BshV$ FLOPs in total.

Thus, for a transformer model with l transformer layers, the total number of floating-point operations is:

$$96Bslh^2\left(1 + \frac{s}{6h} + \frac{V}{16lh}\right).$$

Evaluation

- Comparison to ZeRO-3 (an older version of Microsoft DeepSpeed)
- The results provide a point of comparison to a method that does not use model parallelism. We note that we have only considered ZeRO-3 without tensor parallelism. ZeRO-3 can be combined with model parallelism to potentially improve its scaling behavior.

Computed using the following formula

$$\text{End-to-end training time} \approx \frac{8TP}{nX}.$$

Scheme	Number of parameters (billion)	Model-parallel size	Batch size	Number of GPUs	Microbatch size	Achieved teraFlOP/s per GPU	Training time for 300B tokens (days)
ZeRO-3 without Model Parallelism	174.6	1	1536	384	4	144	90
				768	2	88	74
				1536	1	44	74
	529.6	1	2560*	640	4	138	169
			2240	1120	2	98	137
				2240	1	48	140
PTD Parallelism	174.6	96	1536	384	1	153	84
				768	1	149	43
				1536	1	141	23
	529.6	280	2240	560	1	171	156
				1120	1	167	80
				2240	1	159	42

Table 2: Comparison of PTD Parallelism to ZeRO-3 (without model paralllelism). The 530-billion-parameter GPT model did not fit on 560 GPUs when using a microbatch size of 4 with ZeRO-3, so we increased the number of GPUs used to 640 and global batch size to 2560 to provide a throughput estimate (relevant row marked in table with a *).

Evaluation

Pipeline parallelism

- Weak-scaling performance of pipeline parallelism in isolation.
- As we increase the number of pipeline stages, we also increase the size of the model by proportionally increasing the number of layers in the model.
- We vary the total number of A100 GPUs used from 8 to 64.
- Microbatch size is 1.
- The higher batch size scales better since the pipeline bubble is amortized over more microbatches.

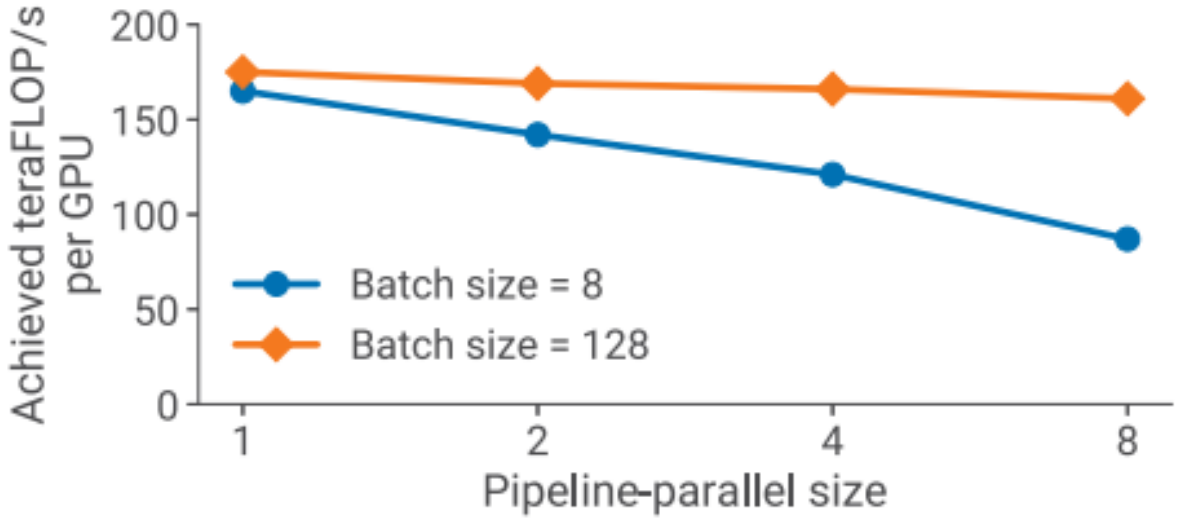


Figure 11: Throughput per GPU of pipeline parallelism using two different batch sizes in a weak-scaling experiment setup (model size increases with the pipeline-parallel size).

Evaluation

Pipeline parallelism

- We compare the performance of the non-interleaved schedule to the interleaved schedule.
- The model is fixed.
- The interleaved schedule with the scatter/gather communication optimization has higher computational performance than the non-interleaved (default) schedule.
- The amount of point-to-point communication within the pipeline is proportional to the batch size, and consequently the non-interleaved schedule catches up as the amount of communication increases (the interleaved schedule features more communication per sample).

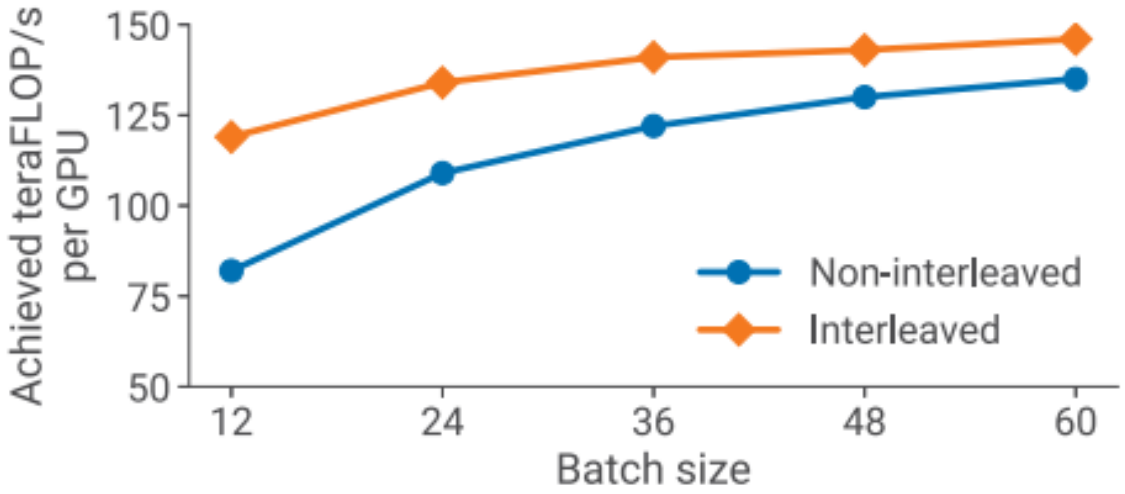


Figure 12: Throughput per GPU of interleaved and non-interleaved schedules for a GPT model (175 billion parameters) on 96 GPUs.

Evaluation

Comparison of parallel configurations

- Performance for parallel configurations using the same number of GPUs for a given model and multiple batch sizes.
- Tensor versus pipeline parallelism for a given model and batch size. All configurations use both parallelism techniques, however, at different degrees.

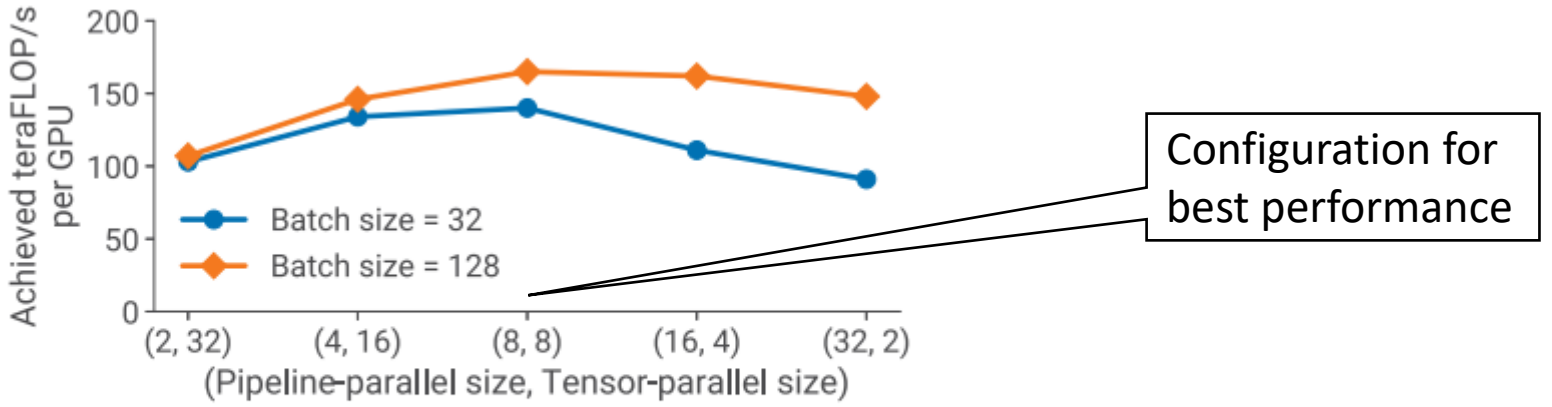


Figure 13: Throughput per GPU of various parallel configurations that combine pipeline and tensor model parallelism using a GPT model with 162.2 billion parameters and 64 A100 GPUs.

Evaluation

Observations: tensor versus pipeline parallelism

- Tensor model parallelism is best within a node (DGX A100 server) due to its expensive all-reduce communication.
- Pipeline model parallelism, on the other hand, uses much cheaper point-to-point communication that can be performed across nodes without bottlenecking the entire computation.
- However, with pipeline parallelism, significant time can be spent in the pipeline bubble: the total number of pipeline stages should thus be limited so that the number of microbatches in the pipeline is a reasonable multiple of the number of pipeline stages (#pipeline stages = #GPUs).
- Neither tensor model parallelism (used by the originl Megatron [40]) nor pipeline model parallelism (used by PipeDream [30] and others) in isolation can match the performance of using both techniques in conjunction.

Evaluation

Pipeline versus data parallelism

- We evaluate the impact of data and pipeline model parallelism on performance.
- For simplicity, we keep the microbatch size equal to 1 in these experiments.
- Observations
  - For each batch size, the throughput decreases as the pipeline parallel size increases, matching our analytical model from §3.3.
  - Pipeline model parallelism should be used primarily to support the training of large models that do not fit on a single worker.
  - Data parallelism should be used to scale up training.

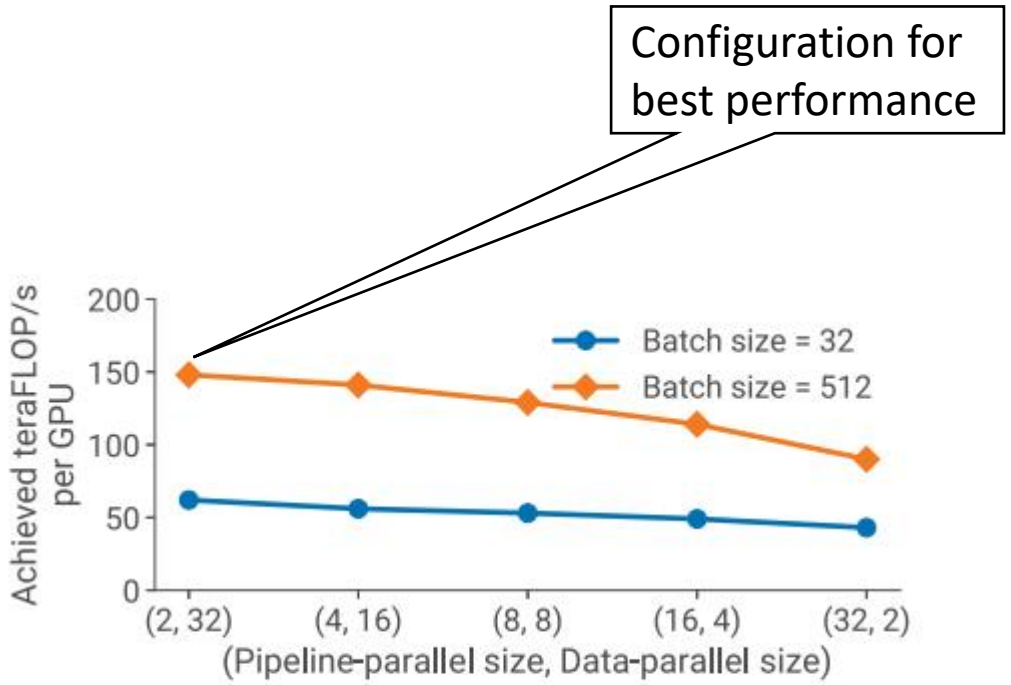


Figure 14: Throughput per GPU of various parallel configurations that combine data and pipeline model parallelism using a GPT model with 5.9 billion parameters, three different batch sizes, microbatch size of 1, and 64 A100 GPUs.

Evaluation

Tensor versus data parallelism

- We evaluate the impact of data and tensor model parallelism on performance.
- For simplicity, we keep the microbatch size equal to 1 in these experiments.
- Observations
  - With larger batch sizes and a microbatch size of 1, data-parallel communication is infrequent.
  - The all-to-all communication required in tensor model parallelism needs to be performed for *every* microbatch in a batch. This all-to-all communication with tensor model parallelism dominates end-to-end training time.
  - As the tensor-model-parallel size increases, we perform smaller matrix multiplications on every GPU, decreasing utilization on each GPU.

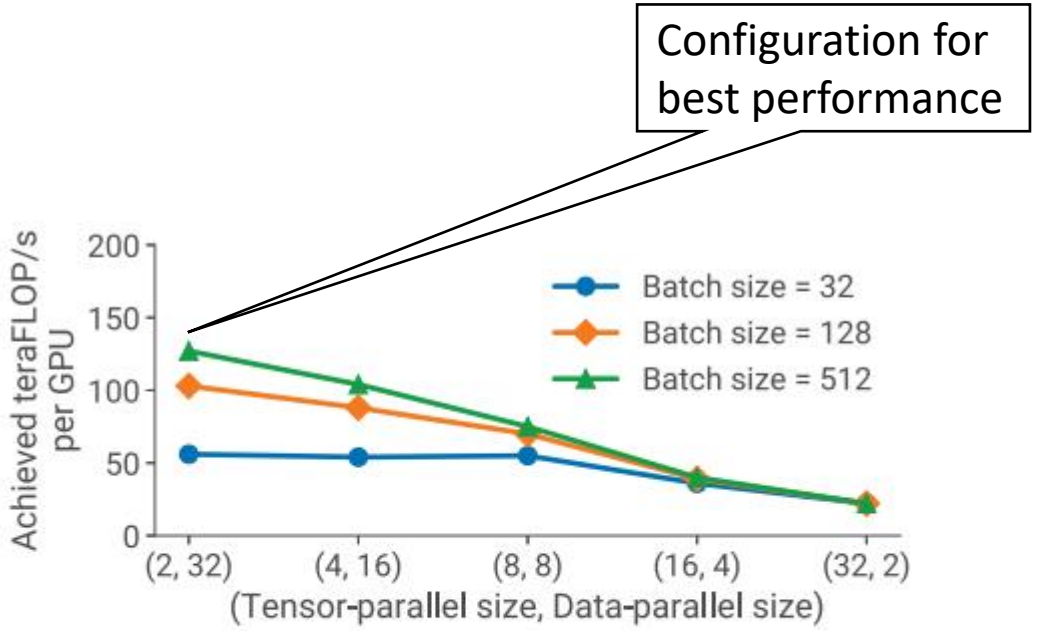


Figure 15: Throughput per GPU of various parallel configurations that combine data and tensor model parallelism using a GPT model with 5.9 billion parameters, three different batch sizes, microbatch size of 1, and 64 A100 GPUs.

Note. GPT-3 was trained to convergence with a batch size of 1,536. Data parallelism thus supports parallelization to only 1,536 GPUs; however, roughly 10,000 GPUs were used to train this model in a reasonable amount of time.

Evaluation

Microbatch size

- For a given batch size, increasing the microbatch size decreases the number of microbatches in the pipeline (m), leading to a larger pipeline bubble.
- Increasing the microbatch size can also improve GPU utilization by increasing the arithmetic intensity of executed kernels.
- This suggests the choice of optimal microbatch size challenging.

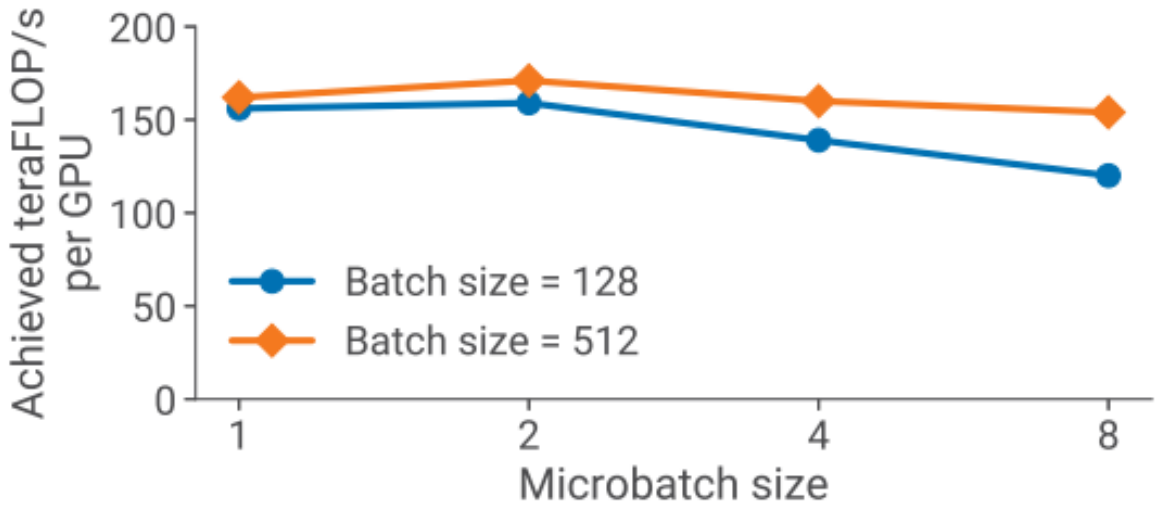


Figure 16: Throughput per GPU of a $(t, p) = (8, 8)$ parallel configuration for different microbatch sizes on a GPT model with 91 billion parameters, for two different batch sizes using 64 A100 GPUs.

Evaluation

Summary

- GPU utilization improves as the models get larger (larger matrix multiplications) without significant increase in the communication time relative to computation time.
- PTD-P achieve 52% of peak device throughput for the largest model, and 44% of peak device throughput for the smallest model.

- Activation recomputation: throughput at large batch sizes with activation recomputation is up to 2× higher due to a smaller pipeline bubble.
- Scatter-Gather optimization: reduces the amount of communication over cross-node links.
- Fused operators: improves GPU throughput (FLOP/s).
- Checkpoint loading and saving: trillion-parameter model has a checkpoint of size 13.8 terabytes.